

EduOne Admin Panel

Backend Implementation

54 API Endpoints

22 New Files

8 Modules

Project: EduOne — AI-Powered Academic Platform

Date: May 7, 2026

Author: AI-Assisted Development

1. Executive Summary

This report documents the complete backend implementation of the **EduOne Admin Panel** — a comprehensive administration system built on top of the existing EduOne academic platform. The admin panel provides 54 RESTful API endpoints across 8 functional modules, enabling platform administrators to manage users, moderate content, monitor feature usage, view analytics, and communicate with users.

Key Deliverables: 22 new files created, 4 existing files modified, 54 API endpoints implemented, 5 new database models, 3 new services, 1 admin middleware, and 9 route files.

2. Architecture Analysis

Before writing any code, a thorough analysis of the existing EduOne backend was performed to ensure the new admin module follows identical conventions:

Aspect	Existing Pattern	Admin Implementation
Framework	Express 5 with ES Modules	Same — <code>import/export</code>
Database	MongoDB via Mongoose 8	Same — 5 new Mongoose schemas
Authentication	JWT (access + refresh tokens)	Same + Admin role check
Response Format	<code>ApiResponse(statusCode, data, message)</code>	Same class used in all controllers
Error Handling	<code>ApiError(statusCode, message)</code>	Same class used everywhere
Async Wrapper	<code>asyncHandler()</code>	Same wrapper on all handlers
File Naming	<code>entity.type.js</code> (camelCase)	Same — e.g., <code>adminAuth.controller.js</code>
Route Pattern	<code>router.route("/path").method(middleware, handler)</code>	Same with inline comments
Real-time	Socket.IO via <code>getIO()</code>	Added admin dashboard room

3. Why an Admin Panel?

EduOne is an AI-powered academic platform with features like Notes Organizer, Academic Planner, Resume Analyzer, and Community Posts. As the platform scales, administrators need:

- **User Management:** Monitor registrations, suspend bad actors, change roles, track activity
- **Content Moderation:** Review flagged community posts, handle reports, remove inappropriate content
- **Feature Monitoring:** Track how Notes, Tasks, and Resumes features are being used
- **Analytics:** Understand user growth, daily/monthly active users, feature adoption
- **Communication:** Send announcements, notifications, and broadcast emails to users
- **System Health:** Monitor server uptime, memory usage, error logs

4. New Database Models

4.1 Report Model — `report.model.js`

Why: The existing community feature had no flagging/reporting mechanism. Users need the ability to report spam, harassment, or inappropriate content. Admins need a queue to review and resolve these reports.

Field	Type	Purpose
reporterId	ObjectId → User	Who filed the report
reportedContentId	ObjectId	The post/comment/user being reported
contentType	Enum: Post, Comment, User	What type of content is reported
reason	Enum: Spam, Harassment, etc.	Categorized reason for the report
status	Enum: Pending, Resolved, Rejected	Admin resolution status
resolvedBy	ObjectId → User	Which admin handled it
adminNotes	String	Admin's notes on the resolution

4.2 Announcement Model — `announcement.model.js`

Why: Admins need to broadcast important messages (maintenance windows, new features, policy changes) to specific audiences. The model supports pinning and expiration for time-sensitive announcements.

4.3 Notification Model — `notification.model.js`

Why: Individual or targeted notifications to specific users or groups. Includes read tracking so admins can monitor notification engagement.

4.4 LoginHistory Model — `loginHistory.model.js`

Why: Security audit trail. Tracks every login attempt (successful and failed) with IP address and user agent. Essential for detecting suspicious activity and providing admin visibility into user access patterns.

4.5 ErrorLog Model — `errorLog.model.js`

Why: System debugging and monitoring. Captures server errors with stack traces, endpoints, and HTTP methods. Enables admins to identify recurring issues without SSH access to the server.

5. Admin Middleware — `adminAuth.middleware.js`

Why: The existing `verifyJWT` middleware only validates authentication. Admin endpoints require an additional authorization layer that checks:

- Valid JWT token (same as existing auth)
- User role is `"Admin"` — rejects Students and Teachers
- Account status is `"Active"` — prevents suspended admins from accessing the panel

This follows the principle of least privilege and ensures no non-admin user can access admin APIs even if they somehow obtain a valid JWT.

6. Module-by-Module Breakdown

6.1 Admin Authentication & Dashboard (4 APIs)

Why separate admin login? While the auth mechanism is identical (JWT), the admin login additionally validates role, logs login attempts to the LoginHistory model, and returns admin-specific context.

Method	Endpoint	Purpose
POST	<code>/api/admin/login</code>	Admin-only login with role validation and audit logging
GET	<code>/api/admin/profile</code>	Get admin's own profile
PUT	<code>/api/admin/change-password</code>	Change admin password
GET	<code>/api/admin/dashboard-overview</code>	Aggregated platform statistics for the dashboard home

The **dashboard overview** endpoint aggregates counts from all collections (Users, Posts, Comments, Notes, Tasks, Resumes, Reports) in a single API call using `Promise.all()` for performance.

6.2 User Management (13 APIs)

Why 13 endpoints? User management is the core of any admin panel. Each endpoint serves a distinct administrative function:

Method	Endpoint	Purpose
GET	/api/admin/users	Paginated list with search, role/status filters, sorting
GET	/api/admin/users/:id	Full user detail view
POST	/api/admin/users/create	Manual user creation (e.g., creating teacher accounts)
PUT	/api/admin/users/:id/update	Edit user details
PATCH	/api/admin/users/:id/suspend	Suspend violating users
PATCH	/api/admin/users/:id/activate	Reactivate suspended users
DELETE	/api/admin/users/:id/delete	Soft delete (sets status to "Deleted")
PATCH	/api/admin/users/:id/role	Promote/demote user roles
GET	/api/admin/users/:id/activity	Cross-feature activity summary
GET	/api/admin/users/:id/login-history	Security audit for a specific user
GET	/api/admin/users/export	CSV download of user data
GET	/api/admin/users/online	Currently active users (in-memory tracker)
POST	/api/admin/users/bulk-action	Batch operations on multiple users

Safety guards: Admins cannot suspend, delete, or change the role of their own account — preventing accidental self-logout.

6.3 Community & Content Moderation (10 APIs)

Why: Reuses the existing Post, Comment, and Bookmark models from the merged community feature. No duplicate models were created. The admin moderation layer adds report handling, flagged content querying, and community analytics.

Method	Endpoint	Purpose
GET	/api/admin/posts	All posts with admin-level filters
GET	/api/admin/posts/:id	Post detail with report count
DELETE	/api/admin/posts/:id/delete	Hard delete with cascade (comments, bookmarks, reports)
GET	/api/admin/posts/flagged	Posts with pending reports
GET	/api/admin/reports	All user reports with filters

GET	/api/admin/reports/:id	Report detail with reported content
PATCH	/api/admin/reports/:id/resolve	Mark report as resolved
PATCH	/api/admin/reports/:id/reject	Reject false reports
DELETE	/api/admin/comments/:id/delete	Remove inappropriate comments
GET	/api/admin/community/stats	Engagement analytics, top contributors, trends

6.4 Notes Management (4 APIs)

Why: Admins need visibility into how the Notes Organizer feature is being used. The delete endpoint handles Cloudinary file cleanup and Qdrant vector database chunk deletion.

Method	Endpoint	Purpose
GET	/api/admin/notes	All users' notes with subject filter and search
GET	/api/admin/notes/:id	Note detail with user info
DELETE	/api/admin/notes/:id/delete	Delete with Cloudinary + Qdrant cleanup
GET	/api/admin/notes/usage-stats	Per-subject distribution, upload rates, top users

6.5 Tasks Management (4 APIs)

Why: Monitor the Academic Planner feature usage including AI scheduling performance metrics like on-time completion rates and priority-based analytics.

Method	Endpoint	Purpose
GET	/api/admin/tasks	All tasks with priority and completion filters
GET	/api/admin/tasks/stats	Completion rates, priority distribution, trends
GET	/api/admin/tasks/ai-performance	AI scheduling analytics and on-time rates
DELETE	/api/admin/tasks/delete/:id	Admin delete

6.6 Resume Management (4 APIs)

Why: Track Resume Analyzer adoption, score distributions, popular job roles, and experience level demographics.

Method	Endpoint	Purpose
--------	----------	---------

GET	/api/admin/resumes	All resumes with experience level filter
GET	/api/admin/resumes/:id	Resume detail with analysis results
DELETE	/api/admin/resumes/:id/delete	Delete with Cloudinary cleanup
GET	/api/admin/resumes/stats	Score distribution, job roles, upload trends

6.7 Analytics Dashboard (8 APIs)

Why: Data-driven decision making. Uses MongoDB aggregation pipelines for scalable analytics.

Method	Endpoint	Purpose
GET	/api/admin/analytics/daily-users	DAU from login history aggregation
GET	/api/admin/analytics/monthly-users	MAU growth trend
GET	/api/admin/analytics/feature-usage	Cross-feature comparison
GET	/api/admin/analytics/user-growth	Registration trends over months
GET	/api/admin/analytics/top-features	Most used features ranked
GET	/api/admin/analytics/traffic-source	Browser distribution from user agents
GET	/api/admin/analytics/error-logs	System error history
GET	/api/admin/analytics/system-health	Server uptime, memory, DB status

6.8 Announcements & Notifications (8 APIs)

Why: Admin-to-user communication channel. Supports targeted audiences, email broadcasts using existing SMTP, and a placeholder for future push notifications.

Method	Endpoint	Purpose
GET	/api/admin/announcements	List announcements (pinned first)
POST	/api/admin/announcements/create	Create with audience targeting
PUT	/api/admin/announcements/:id/update	Edit announcement
DELETE	/api/admin/announcements/:id/delete	Remove announcement
POST	/api/admin/notifications/send	Send to specific users
POST	/api/admin/email/broadcast	Bulk email via SMTP (batched)

POST	/api/admin/push-notification/send	Placeholder for Firebase/OneSignal
PATCH	/api/admin/announcement/pin/:id	Toggle pin status

7. Services Created

7.1 `adminAnalytics.service.js`

Why: Separates complex MongoDB aggregation pipelines from controllers. Functions like `aggregateDAU()`, `aggregateMAU()`, and `aggregateUserGrowth()` are reusable and can be consumed by future dashboard widgets or scheduled reports.

7.2 `adminExport.service.js`

Why: CSV generation utility for user data export. Handles date formatting, escaping special characters, and null values. Follows a reusable pattern for future export features.

7.3 `activeUsersTracker.service.js`

Why: In-memory Map-based tracker for online user presence, consistent with the existing `sessionManager.js` pattern. Uses a 5-minute TTL with automatic cleanup. Lightweight alternative to a Redis-based solution.

8. Existing File Modifications

File	Change	Reason
<code>app.js</code>	Added admin router import and mount at <code>/api/admin</code>	Register all admin routes with the Express app
<code>user.model.js</code>	Added <code>lastLoginAt</code> field	Track when users last logged in for admin monitoring
<code>user.controller.js</code>	Record <code>lastLoginAt</code> on login	Populate the new field on each successful login
<code>socket.config.js</code>	Added <code>joinAdminDashboard</code> event	Enable real-time admin dashboard room for future live updates

9. File Structure

```

src/
├── controllers/admin/           (8 files - NEW)
│   ├── adminAuth.controller.js
│   ├── adminUser.controller.js
│   ├── adminCommunity.controller.js
│   ├── adminNotes.controller.js
│   ├── adminTasks.controller.js
│   ├── adminResumes.controller.js
│   ├── adminAnalytics.controller.js
│   └── adminAnnouncement.controller.js
├── routes/admin/              (9 files - NEW)
│   ├── index.routes.js
│   ├── adminAuth.routes.js
│   ├── adminUser.routes.js
│   ├── adminCommunity.routes.js
│   ├── adminNotes.routes.js
│   ├── adminTasks.routes.js
│   ├── adminResumes.routes.js
│   ├── adminAnalytics.routes.js
│   └── adminAnnouncement.routes.js
├── models/                   (5 files - NEW)
│   ├── report.model.js
│   ├── announcement.model.js
│   ├── notification.model.js
│   ├── loginHistory.model.js
│   └── errorLog.model.js
├── middlewares/
│   └── adminAuth.middleware.js (NEW)
├── services/                 (3 files - NEW)
│   ├── adminAnalytics.service.js
│   ├── adminExport.service.js
│   └── activeUsersTracker.service.js
└── admin.constants.js        (NEW)

```

10. Design Decisions

- **Soft Delete for Users:** Sets `accountStatus` to "Deleted" instead of removing from DB — preserves data integrity and allows recovery.
- **Separate LoginHistory Model:** Instead of embedding login records in the User document (which would grow unboundedly), a separate collection scales independently.
- **In-Memory Online Users:** Chose a Map-based tracker over database polling for real-time performance. Trade-off: data lost on server restart, which is acceptable for presence tracking.
- **Batch Email with Backpressure:** Email broadcast sends in batches of 10 to avoid overwhelming the SMTP server.
- **Cascading Deletes:** Admin post deletion also removes related comments, bookmarks, and auto-resolves pending reports.

- **Self-Protection Guards:** Admins cannot suspend, delete, or change their own role to prevent accidental logout.

11. Verification

All 27 files (22 new + 5 modified) passed Node.js syntax validation (`node --check`). The implementation follows the exact same patterns, naming conventions, and architectural decisions as the existing EduOne codebase.